

MODULE 2: Transport & Network Layers with Linux Kernel

PCCST501 Computer Networks | Quick Exam Revision Guide | KTU S5 CSE | Prof. Anju Markose, VJCET

1. Transport Layer: UDP & TCP Protocols

Transport Layer Role: Logical process-to-process communication using 16-bit Port Numbers (0-65535).

- **UDP (User Datagram Protocol):** Connectionless, unreliable, 8-byte fixed header (Src Port, Dst Port, Length, Checksum). Uses 1's complement addition over pseudo-header + UDP header + payload.
- **TCP (Transmission Control Protocol):** Connection-oriented, reliable byte stream, full-duplex, flow-controlled, congestion-controlled.
- **TCP Header (20-60 Bytes):** Sequence Number, Acknowledgment Number, Header Length (Data Offset), Flags (URG, ACK, PSH, RST, SYN, FIN), Advertised Window (rwnd), Checksum, Urgent Pointer.
- **Connection Setup & Tear-down:** 3-Way Handshake (SYN → SYN-ACK → ACK) and 4-Way Handshake (FIN → ACK, FIN → ACK).

Key Concept / Exam Formula:

TCP Congestion Control Phases:

1. **Slow Start:** cwnd starts at 1 MSS, doubles every RTT (exponential growth: 1, 2, 4, 8...) until reaching ssthresh.
2. **Congestion Avoidance (AIMD):** When cwnd $\geq$ ssthresh, cwnd increases linearly by 1 MSS per RTT.
3. **On Loss Detection:**
 - *Timeout (Severe):* ssthresh = cwnd/2, cwnd = 1 MSS → enters Slow Start.
 - *3 Duplicate ACKs (Mild):* ssthresh = cwnd/2. In **TCP Reno**, cwnd = ssthresh + 3 MSS (Fast Recovery).

2. Socket Programming & I/O Multiplexing (Hands-on)

TCP Socket Lifecycle:

- **Server:** socket() → bind() → listen() → accept() → read()/write() → close()
- **Client:** socket() → connect() → write()/read() → close()

I/O Multiplexing (select vs poll):

- **select():** Monitors bit-masks of file descriptors (readfds, writefds) up to FD_SETSIZE (1024). Reconstructs fd_set on each loop.
- **poll():** Uses an array of struct pollfd (fd, events, revents). Eliminates the 1024 fd limit and avoids fd_set reconstruction.

3. Network Layer: IPv4, Subnetting & Unicast Routing

IPv4 Datagram Header (20-60 Bytes): Ver (4b), IHL (4b), Type of Service (8b), Total Length (16b), ID (16b), Flags [DF, MF] (3b), Fragment Offset (13b), TTL (8b), Protocol [TCP=6, UDP=17] (8b), Header Checksum (16b), Src IP (32b), Dst IP (32b).

Key Concept / Exam Formula:**Subnetting Calculations:**

Given prefix /n → Subnet mask has n leading 1s. Host bits = 32 - n.

- Number of usable hosts = $2^{(32 - n)} - 2$
- Network ID = IP bitwise AND Subnet Mask
- Directed Broadcast Address = IP with all host bits set to 1

Unicast Routing Protocols:

- **Distance Vector (RIP):** Based on Bellman-Ford algorithm. Metric = Hop count (Max 15 hops, 16 = infinity). Suffers from *Count-to-Infinity* (mitigated by Split Horizon & Poison Reverse).
- **Link State (OSPF):** Based on Dijkstra's Shortest Path First (SPF) algorithm. Floods Link State Advertisements (LSAs) within hierarchical Areas (Area 0 = Backbone). Metric = Cost (Bandwidth based).
- **Path Vector (BGP):** Inter-Autonomous System (Inter-AS) protocol. Exchanges complete AS-Path vectors to eliminate loops and enforce routing policies.

4. Multicast Routing, Next-Gen IP (IPv6) & QoS

- **Multicast Protocols:** IGMP (group joins/leaves). DVMRP (Reverse Path Forwarding with flood & prune), PIM-DM (dense mode flood/prune), PIM-SM (sparse mode Rendezvous Point shared tree).
- **IPv6 Enhancements:** 128-bit addresses (hex: 2001:db8::1), fixed 40-byte base header (no checksum, faster processing), built-in security (IPsec), SLAAC stateless auto-configuration.
- **QoS Traffic Shaping:** *Leaky Bucket* (enforces strict constant rate, discards excess bursts) vs *Token Bucket* (tokens accumulate periodically, allowing burst transmission up to token capacity).
- **Linux Routing Table Hands-on:** Kernel FIB (Forwarding Information Base). Command: `ip route add 192.168.10.0/24 via 10.0.0.1 dev eth0`, `ip route show`.